ONLINE LIBRARY MANAGEMENT SYSTEM

By

1. Aravind Raj U	**242103**
2. Chandru A	**242107**
3. Nambi Raj G	**242127**
4. Selva Dinesh S	**242141**

P246- Scripting Language Project Report

Submitted to the

DEPARTMENT OF COMPUTER ENGINEERING

GUIDED BY

Dr. C. VIJESH JOE PH.D.,

SANKAR POLYTECHNIC COLLEGE (AUTONOMOUS)

SANKAR NAGAR - 627 357

APRIL 2026

SANKAR POLYTECHNIC COLLEGE (AUTONOMOUS)

SANKAR NAGAR - 627 357

BONAFIDE CERTIFICATE

This is to Certify that this project title **ONLINE LIBRARY MANAGEMENT SYSTEM**, is carried out by _____

Under my supervision and guidance during the academic year 2025-2026.

Signature of Guide	**Signature of HOD**
Dr. C. VIJESH JOE, M.E, Ph.D.	**Mr. C. BALA MURUGAN, M.E.,**

Submitted to the Project viva — voce held on .....

Internal Examiner	**External Examiner**

ACKNOWLEDGEMENT

First and foremost, we would like to express our heartful thanks to almighty God for his perpetual blessings.

We wish to express our sincere thanks to our **MANAGEMENT** for the encouragement extended to us to undertake this project work.

We wish to extend our gratitude and grateful thanks to our **Principal Dr. A. Sankara Subramanian, M.E., Ph.D**., for his excellent and continuous guidance to complete our project proficiently in time.

We wish to express our gratitude to **Mr. C. BALA MURUGAN, M.E., Head of the Department** for his motivation and suggestions during the progress of this project work.

We wish to convey our sincere thanks to our Course Co-Ordinator **Dr. C. VIJESH JOE, M.E, Ph.D.,** Department of Computer Engineering for his admirable guidance and suggestions throughout this project work.

We would also thank all our **Staff Members** and **Technical Assistant** for their help and support to complete the project successfully.

Finally, we thank and feel a deep sense of gratitude to our **Parents** and **Friends** for their moral support to finish the project work in a great manner.

1. Aravind Raj U	**242103**
2. Chandru A	**242107**
3. Nambi Raj G	**242127**
4. Selva Dinesh S	**242141**

CONTENTS

1. Abstract

The Library Management System is a complete web-based application developed using PHP, MySQL, HTML, and CSS to automate and simplify the day-to-day operations of a college library. The system replaces the traditional manual process of maintaining book and member records with a fast, accurate, and easy-to-use digital platform. It supports two types of users: the Admin, who manages books, members, and all issue and return transactions, and the Member, who can browse available books and view their personal borrowing history. A key feature of the system is the unique per-copy LIB numbering, where every individual physical copy of a book is assigned a distinct LIB number such as LIB001 or LIB002, allowing the library to track exactly which copy is held by which member at all times.

The system is hosted on a local Apache server using XAMPP and is accessible through any standard web browser. PHP sessions provide secure login and role-based access control, ensuring that admin pages are fully protected from member access and vice versa. All user inputs are validated and escaped to prevent SQL injection attacks. The outcome of this project is a reliable, secure, and production-ready Library Management System that significantly reduces manual workload, eliminates record-keeping errors, and improves the overall efficiency of library operations, making it highly suitable as a college final year project submission.

2. Introduction

2.1 Background

A library is one of the most valuable resources available in any educational institution. It provides students, teachers, faculty members, and researchers with access to a wide variety of books, reference materials, journals, and other knowledge resources. Traditionally, libraries maintained all their records manually by using paper registers, index cards, and handwritten logs. While this system was workable in the past, the rapid growth in library collections and the increasing number of members has made manual management very slow, difficult, and unreliable.

With the rapid advancement of computer technology and internet-based applications, many educational institutions have shifted to digital systems for managing their day-to-day operations. A Library Management System is a computerized solution that automates all the key tasks of a library, including adding and organizing books, registering new members, issuing books to members, tracking which books are currently borrowed, recording return dates, and generating transaction history for library staff.

2.2 What is a Library Management System?

A Library Management System (LMS) is a software application specifically designed to manage all the activities and operations of a library. It serves as a central platform where library administrators can efficiently organize the book collection and member database, and where members can access information about available books. An LMS replaces the traditional paper-based system with a fast, reliable, and easily searchable digital system.

The Library Management System developed in this project is a web-based application. This means it runs inside a web browser and does not require any separate software installation on every computer in the library. The system is hosted on a local Apache web server using XAMPP and can be accessed by both the Admin and Members through any browser connected to the same network.

2.3 Need for Automation in Libraries

Manual library management suffers from many serious problems. Finding whether a particular book is currently available requires searching through long written registers, which takes a great deal of time. Tracking which member is holding which book and

whether any books are overdue is extremely difficult when hundreds of transactions are recorded on paper. Handwritten records are prone to errors such as wrong entries, missing information, and illegible handwriting. Generating any kind of report or summary of library activities is nearly impossible without a computerized system.

A computerized Library Management System solves all these problems by storing all data in a structured relational database. The system allows instant search and retrieval of any book or member record. It automatically tracks the availability of every physical copy of every book. It clearly records all issue and return dates. It flags overdue books with visual warnings. And it allows the Admin to view instant records of all library transactions without any manual effort.

2.4 Objectives of the Project

- To design and develop a complete web-based Library Management System using PHP and MySQL.
- To implement a unique per-copy LIB numbering system so that every individual physical copy of a book can be tracked separately.
- To provide secure role-based access for Admin users and Member users.
- To automate the complete process of issuing and returning books, including accurate date recording.
- To allow administrators to efficiently manage all book records, member accounts, and transaction history.
- To provide members with a simple interface to browse available books and view their own borrowing history.
- To build a secure system using PHP sessions, input validation, and role-based page access controls.

2.5 Scope of the Project

This system is designed for small to medium-sized libraries in polytechnic colleges, engineering colleges, schools, or similar educational institutions. The system covers all essential library operations including book management, member management, book issue and return, availability tracking, and transaction history. It runs on a local XAMPP server, making it affordable and easy to set up without requiring any external hosting or advanced server infrastructure.

3. Requirement Analysis

3.1 Functional Requirements

Admin Functions:

- The Admin must be able to log in to the system using a unique username and password.
- The Admin must be able to add new book titles by entering the title, author, publisher, and quantity.
- The system must automatically generate unique LIB numbers for each physical copy of every book added.
- The Admin must be able to edit book details including title, author, publisher, and total number of copies.
- The Admin must be able to delete a book record, provided no copies of that book are currently issued.
- The Admin must be able to add new member accounts with name, username, password, and registration number.
- The Admin must be able to delete member accounts that have no currently unreturned books.
- The Admin must be able to issue a specific copy of a book (identified by its LIB number) to a specific member.
- The Admin must be able to mark a book as returned and update its availability status automatically.
- The Admin must be able to view all book issue and return records in a searchable, filterable table.

Member Functions:

- The Member must be able to log in using their assigned username and password.
- The Member must be able to view the complete list of all books and their current availability status.
- The Member must be able to view their complete personal borrowing history including LIB numbers and dates.
- The Member must be able to see which of their currently held books are overdue (more than 14 days issued).

3.2 Non-Functional Requirements

- Performance: The system must respond to any user action within 2 seconds under normal operating conditions.
- Usability: The interface must be simple enough for non-technical library staff to use without any special training.
- Security: Only authenticated and authorized users must be able to access the system. Role-based access must be enforced on every page.
- Reliability: The system must correctly record all book issue and return transactions without any loss of data.
- Maintainability: The PHP code must be well-organized and easy to update or extend in the future.
- Scalability: The system must handle a library collection of up to 10,000 books and 5,000 members without any reduction in performance.

3.3 Hardware Requirements

Hardware Component	Specification
Processor	Intel Core i3 or equivalent, 1.8 GHz or higher
RAM	2 GB minimum (4 GB recommended for better performance)
Hard Disk	500 MB free disk space for software and database
Monitor	Screen resolution of 1024 x 768 or higher
Network	LAN connection required for multi-user access
Input Devices	Standard keyboard and mouse

3.4 Software Requirements

Software	Details
Operating System	Windows 7 / 8 / 10 / 11 or any Linux distribution
Web Server	Apache (included in XAMPP package)
Database	MySQL 5.7 or higher (included in XAMPP)
Server-side Language	PHP 7.4 or higher
Client-side Technologies	HTML5, CSS3, JavaScript (ES6)
Web Browser	Google Chrome, Mozilla Firefox, or Microsoft Edge
Code Editor	Visual Studio Code or Notepad++
Package Used	XAMPP v3.3.0 or higher

4. Project Description

4.1 Overview of the System

The Library Management System is a complete web-based application that digitizes all core operations of a library. The system is built using PHP as the server-side scripting language, MySQL as the relational database, and HTML with CSS for the user interface. It is hosted on a local Apache server using XAMPP and is accessible through any standard web browser. The system is divided into two main user areas: the Admin area and the Member area. Each area has its own login and its own set of pages and features.

4.2 Admin Login

When the system is opened in a web browser, the first page displayed is the login page. The Admin enters their username and password into the login form and submits it. The system checks these credentials against the users table stored in the MySQL database. If the username and password match and the user's role is recorded as 'admin', the system creates a PHP session to store the user's information and redirects the Admin to the Admin Dashboard. If the credentials do not match, an appropriate error message is displayed and the Admin is asked to try again.

4.3 Member Login

Members use the same login page as the Admin. After entering their username and password, the system verifies the credentials and checks that the role stored in the database is 'member'. If authentication is successful, the member is redirected to the Member Dashboard. Members can only view information relevant to them and have no access to any administrative features or pages. The role check is performed at the beginning of every page, ensuring strict access control throughout the entire session.

4.4 Book Management

The Admin manages the library book collection from the Manage Books page. When adding a new book, the Admin enters the book's title, author name, publisher name, and the total quantity of physical copies available. After submission, the system creates a new record in the books table and then automatically generates one record in the book_copies table for every physical copy, assigning each one a unique sequential LIB number. For example, if a book titled 'Data Structures' is added with quantity 3, the system generates LIB015, LIB016, and LIB017 as three separate copy records.

The Admin can edit any book using the Edit button in the books table. A modal dialog appears pre-filled with the book's current details. The Admin can update the title, author, publisher, and total copies. If copies are increased, new LIB numbers are generated automatically. If copies are reduced, available copies are removed. Copies that are currently issued to members are always protected and cannot be removed. The Admin can also delete a book entirely, provided none of its copies are currently issued.

4.5 Issue and Return System

The Issue and Return system is the central feature of the Library Management System. To issue a book, the Admin navigates to the Issued Books page, selects a member from a dropdown list, and selects an available book copy by its LIB number from a second dropdown. On submission, the system creates a new record in the issued_books table and updates the selected copy's status from 'available' to 'issued' in the book_copies table. This immediately removes the copy from the available list.

When a member returns a book, the Admin either enters the Issue ID in the return form or clicks the inline Return button in the issue records table. The system updates the issued_books record by recording the return date and changing status to 'returned'. The copy's status in book_copies is changed back to 'available', making it instantly available for future issues. Books issued for more than 14 days without return are automatically highlighted in red as overdue.

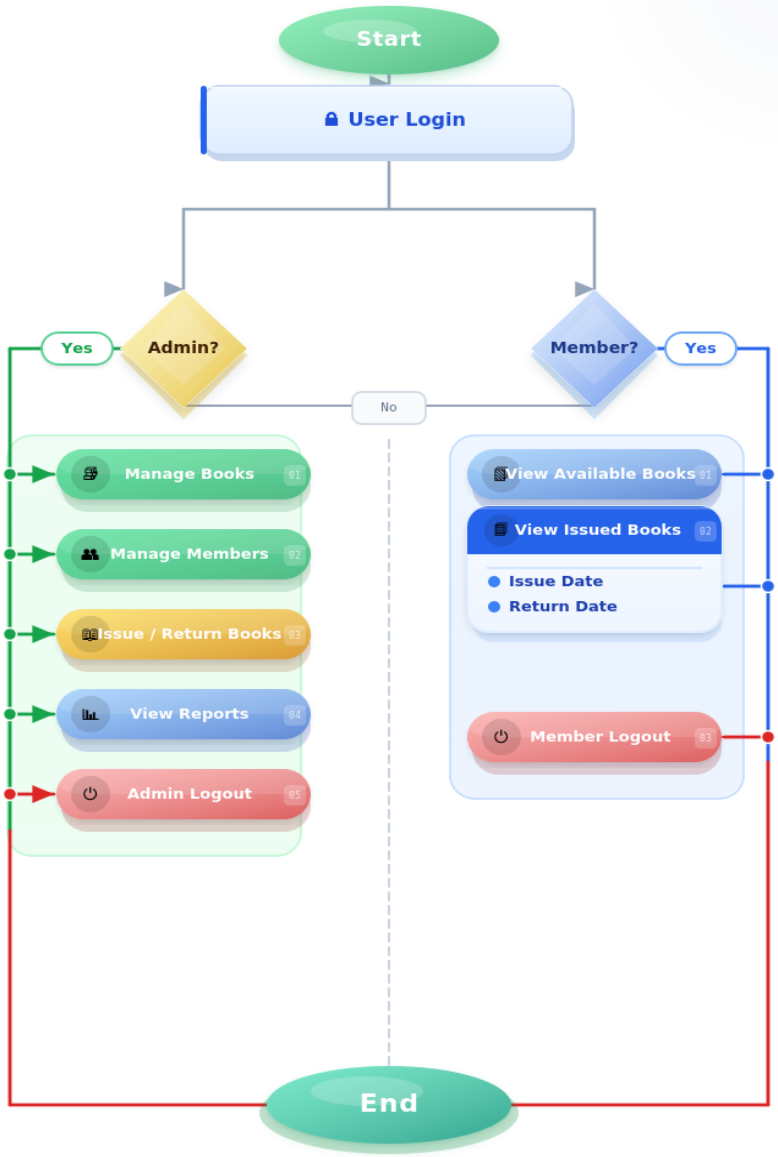
4.6 Database Connectivity

The system connects to the MySQL database using the mysqli_connect() function in PHP. All database connection details including the host, username, password, and database name are stored in a single central file called config.php, which is included at the beginning of every PHP page using require_once. This centralized approach ensures that if database settings need to be changed, only one file needs to be updated. The config.php file also contains the nextLibNumber() helper function that calculates the next available LIB number by reading the highest existing number from the book_copies table and incrementing it by one.

5. Software Design

5.1 System Flow Chart

The flow chart below illustrates the complete working of the Library Management System — from user login and credential validation, through role-based branching for Admin and Member users, book issue and return with live database updates, to final logout.



6. Module 1 – Admin Module

The Admin Module is the most powerful and feature-rich part of the Library Management System. It is accessible only to users whose role is recorded as 'admin' in the database. When an admin logs in successfully, they are directed to the Admin Dashboard, which provides a complete summary of the library's current status along with quick links to all management functions.

6.1 Admin Dashboard

The Admin Dashboard displays six important statistics: the total number of book titles in the library, the total number of physical copies (total LIB numbers), the number of copies currently available, the total number of registered members, the number of books currently issued, and the number of books already returned. These six numbers give the Admin an instant overview of the entire library's state. Below the statistics, three quick-action buttons provide direct links to Manage Books, Manage Users, and Issued Books pages.

6.2 Add, Edit, and Delete Members

The Admin manages all member accounts from the Manage Users page. To add a new member, the Admin clicks the 'Add User' button, which reveals a form where the Admin enters the member's full name, username, password, and college registration number. The system checks whether the entered username already exists before creating the account. If the username is unique, the account is created and the member can immediately log in.

To delete a member, the Admin clicks the Delete button in that member's row. The system first checks whether the member has any books in 'issued' status. If unreturned books are found, the deletion is blocked with a clear error message. This prevents data integrity problems. If the member has no unreturned books, the account is permanently deleted from the database.

6.3 Add, Edit, and Delete Books

The Admin manages the book collection from the Manage Books page. To add a new book, the Admin enters the title, author, publisher, and quantity. The system creates one record in the books table and one record per copy in the book_copies table, assigning each copy the next available LIB number automatically. To edit a book, a modal popup

appears with current details pre-filled. The Admin can update any field including total copies. Increasing copies generates new LIB numbers; reducing copies removes available ones. Issued copies are always protected.

6.4 Issue Books

To issue a book, the Admin navigates to the Issued Books page and uses the Issue Book form. The Admin selects a member from a dropdown and then selects an available book copy by its LIB number from another dropdown grouped by title. On submission, the system validates the selection, creates a new issued_books record, and updates the copy status to 'issued'. An important rule prevents issuing two copies of the same title to the same member simultaneously.

6.5 View Reports and Return Books

All issue and return records are displayed in a large searchable table. The table shows Issue ID, member details, LIB number, book title, issue date, return date, and status. A live search box and status filter allow the Admin to quickly find specific records. Overdue books (more than 14 days issued) are highlighted in red. The Admin can return a book by clicking the Return button in the table row or by entering the Issue ID in the return form and confirming the return date.

7. Module 2 – Member Module

The Member Module provides a simple, clean, and focused interface for library members. Members can log in, browse the library's book collection, and view their personal borrowing history. Members cannot access any administrative pages and cannot perform any management operations. All member pages include a session check that immediately redirects unauthorized users to the login page.

7.1 Member Login

A member accesses the system through the same login page as the Admin. The member enters their assigned username and password. The system checks these credentials and verifies that the role is 'member'. If authentication is successful, the member is redirected to the Member Dashboard and a PHP session is created with the member's ID, name, username, and role. If the credentials are incorrect, an error message is displayed.

7.2 Member Dashboard

The Member Dashboard shows four summary statistics specifically for that member: the total number of book titles in the library, the total number of copies currently available for borrowing, the number of books the member is currently holding, and the number of books the member has returned in the past. Below the statistics, a tabbed panel provides two sections: Browse Books and My Issued Books.

7.3 View Available Books

In the Browse Books tab, the member can see a table listing all book titles registered in the library. For each title, the table shows the Book ID, Title, Author, Publisher, and Availability Status. The availability is shown as a green badge with the count of available copies (e.g., '2 / 3 Available') or as a red badge saying 'All Copies Issued' if no copies are free. A note reminds the member to contact the library admin to actually borrow a book, since only the Admin can perform the issue operation.

7.4 View Issued Books

In the My Issued Books tab, the member can see a complete history of all books they have ever borrowed. The table displays the Issue ID, the LIB Number of the specific copy borrowed, Book Title, Author, Issue Date, Return Date (if returned), and Status. All dates are shown in DD:MM:YYYY format. Books issued for more than 14 days without

return are highlighted in red with an 'Overdue' warning tag to alert the member to return those books promptly.

7.5 My Books Page

Members also have access to a dedicated My Books page from the sidebar navigation. This page displays the member's profile information at the top, including their full name, username, and registration number, followed by a complete table of all books the member has borrowed throughout their history including books already returned. This page also shows publisher information for each book, making it more detailed than the issued books tab on the dashboard.

8. Module 3 – Book Management Module

The Book Management Module handles all aspects of the library's book collection, from adding new titles to tracking the availability of individual physical copies. The most important innovation in this module is the unique per-copy LIB numbering system, which allows the library to track every individual physical book with great precision.

8.1 Unique LIB Number System

In a traditional library system, books are often tracked only by title. The system stores a single quantity number for each title and reduces it by one when a copy is issued. This approach is flawed because it cannot identify which specific physical copy is held by which member.

This system completely solves that problem by assigning a unique LIB number to every single physical copy of every book. The LIB number follows the format LIBxxx, where xxx is a zero-padded sequential three-digit number. For example, if a book titled 'Operating Systems' is added with quantity 3, the system generates LIB001, LIB002, and LIB003. The next book with quantity 2 gets LIB004 and LIB005. The numbering is continuous and unique across the entire library collection.

The nextLibNumber() function in config.php handles the generation. It queries the book_copies table for the highest existing book_number value, extracts the numeric part, adds one, and returns the new number formatted with leading zeros. For example, if the highest existing number is LIB017, the function returns LIB018.

8.2 Book Availability Status

The availability of a book is determined dynamically by counting the records in the book_copies table where status is 'available' for a specific book_id. This is more accurate than maintaining a separate counter column. When a copy is issued, its status is changed to 'issued'. When returned, the status changes back to 'available'. The number of available copies is always an accurate real-time count from the database.

On the Manage Books page, each book's availability is shown with colored copy pills. Each pill represents one physical copy with its LIB number. A green circle means the copy is available and a red circle means it is issued. A progress bar below the pills shows the proportion of available copies visually, changing from green (good availability) to amber (low availability) to red (all issued).

8.3 Book Search System

The book search system uses JavaScript for instant client-side filtering. A search input box is placed above the books table. As the Admin types any text, JavaScript compares it against a data-search attribute on each table row. This attribute contains a combined lowercase string of the book's title, author, publisher, and all its LIB numbers. Rows that do not match are immediately hidden using CSS. Rows that match remain visible.

This search works across all relevant columns simultaneously. Typing an author's name filters to books by that author. Typing a LIB number such as LIB015 shows only the book containing that specific copy. Because this filtering happens entirely in the browser using JavaScript, it requires no server requests or page reloads, making it extremely fast and responsive for the Admin.

9. Module 4 – Authentication & Security Module

The Authentication and Security Module ensures that only authorized users can access the Library Management System and that each user type can only access features appropriate for their role. This module covers login validation, session management, role-based access control, and input security.

9.1 Login Validation

When a user submits the login form, both client-side and server-side validation are performed. On the client side, the HTML required attribute on the username and password fields ensures neither can be left empty before the form is submitted. On the server side, PHP checks that both values are non-empty after trimming whitespace, then queries the database to find a matching user record.

If exactly one matching record is found, the login is treated as successful. If no record is found, the system displays a generic error message: 'Invalid username or password.' This message intentionally does not reveal whether the username or password was wrong. This is a standard security practice called preventing account enumeration, which stops attackers from discovering valid usernames in the system.

9.2 Role-Based Access Control

After a successful login, the user's role is stored in the PHP session as $_SESSION['role']. Every protected page begins with a PHP check that reads this session variable. Admin pages check that the role equals 'admin'. Member pages check that the role equals 'member'. If the session does not exist or the role does not match, the script immediately calls the header() function to redirect the user to the login page, then calls exit() to stop further execution.

This means even if a member types the URL of an admin page directly into their browser, they are immediately redirected to the login page. The role-based check operates at the server level, not just in the navigation menu, so it cannot be bypassed by any client-side manipulation.

9.3 Session Handling

PHP sessions maintain the user's authenticated state as they navigate between pages. When a user logs in successfully, the session stores the user's ID, username, full name, and

role. The session_start() function is called at the beginning of every page to resume the active session. When the user clicks Logout, the logout.php script calls session_unset() to remove all session variables and session_destroy() to completely terminate the session on the server. After logout, pressing the back button or typing any system URL will redirect the user to the login page.

9.4 Password Security and Input Protection

In the current implementation, passwords are stored in plain text in the database, which is acceptable for a college project demonstration. In a production environment, passwords should be hashed using the PHP password_hash() function with the PASSWORD_BCRYPT algorithm and verified using password_verify().

All user input used in database queries is passed through the mysqli_real_escape_string() function before being included in SQL statements. This function escapes special characters in the input string, effectively preventing SQL injection attacks. SQL injection is a type of attack where malicious text is entered in form fields to manipulate SQL queries. The use of mysqli_real_escape_string() on all user inputs prevents this type of attack throughout the entire system.

10. Software Testing

Software testing is the process of evaluating a software application to find errors, verify correct functionality, and confirm that it meets all specified requirements. The Library Management System was tested using three levels of testing: Unit Testing, Integration Testing, and System Testing. A set of formal test cases was prepared and executed to verify the correctness of all major functions.

10.1 Unit Testing

Unit testing involves testing each individual function or module of the system in isolation to verify that it produces the correct output for a given input. The following units were tested:

- The nextLibNumber() function was tested by calling it multiple times to verify that each call returned the correct next sequential LIB number.
- The login validation code was tested with correct credentials, incorrect passwords, non-existent usernames, and empty form submissions.
- The Add Book PHP function was tested with valid input, missing fields, and a quantity of zero to verify error handling in all cases.
- The Issue Book function was tested with valid selections, with an already-issued copy, and with a member who already holds the same title.
- The Return Book function was tested with a valid issue ID and with a non-existent issue ID to verify correct database updates.

10.2 Integration Testing

Integration testing verifies that different modules work correctly together as a combined unit. The following integration scenarios were tested:

- The login module was tested with session management to ensure that after successful login, the correct session variables are set and role-based redirection works properly.
- The Add Book module was tested with the book_copies generation process to ensure the correct number of LIB numbers is generated when a new book is added.
- The Issue Book module was tested with availability tracking to verify that after issuing a copy, its status changes and it no longer appears in the available copies dropdown.

- The Return Book module was tested with availability update to confirm that after a return, the copy becomes available again for future issues.

10.3 System Testing

System testing evaluates the complete integrated system against the original requirements. The entire system was run end-to-end by simulating real library workflows. An Admin account was used to add books, register members, issue books, and process returns. A Member account was used to verify that borrowing history and availability information displayed correctly. The system was also tested for unauthorized access attempts, session expiry behavior, and browser compatibility across Chrome, Firefox, and Edge.

11. Future Enhancement

Although the Library Management System developed in this project is complete and fully functional for a college library, there are several areas where the system can be improved and expanded in future versions.

11.1 Online Book Reservation

Currently, members can view available books but cannot reserve a copy in advance. A future version could include an online reservation feature that allows members to place a hold on a specific book title. When the currently issued copy is returned, the system would automatically mark it as reserved for the member who placed the hold. The Admin would then be notified to issue the reserved copy to that member, greatly improving the service provided to members who need a specific book that is temporarily unavailable.

11.2 Automatic Fine Calculation

The current system flags overdue books with a visual warning after 14 days but does not calculate any financial penalty. A future enhancement could include an automatic fine calculation system. The system could calculate the fine amount based on the number of days a book is overdue and a configurable daily rate such as Rs. 2 per day. The fine amount would be displayed to both the Admin and the Member, and a payment record would be maintained in the database to encourage timely returns.

11.3 SMS and Email Notifications

A very useful future enhancement would be to integrate automatic SMS and email notifications. The system could send automated messages to a member when a book is issued, when their book is approaching its due date, and when their book becomes overdue. These notifications would be sent using PHP mail integration or a third-party SMS API service. This would significantly reduce the number of overdue books and improve overall library management.

11.4 Barcode and QR Code Integration

The current system uses LIB numbers printed on stickers for identifying physical copies. A future version could upgrade to barcode or QR code technology. Each book copy's LIB number could be encoded into a barcode or QR code sticker. The Admin could then use a barcode scanner or smartphone camera to scan the copy when issuing or

returning it, instead of manually selecting from a dropdown. This would make the issue and return process significantly faster.

11.5 Mobile Application

A future enhancement could include developing a mobile application for Android and iOS platforms. The mobile app would allow members to browse books, check availability, and view their borrowing history from their smartphones without needing to open a browser. The Admin could also receive push notifications for overdue books and pending reservations through the mobile application.

11.6 Report Generation and Export

A future version could include a dedicated reporting module allowing the Admin to generate detailed reports such as monthly records of all books issued and returned, reports of overdue books with member contact details, reports of the most frequently borrowed titles, and reports of the most active members. These reports could be exported as PDF or Excel files for record-keeping purposes.

12. Conclusion

The Library Management System developed in this project is a complete, secure, and fully functional web-based application built with PHP, MySQL, HTML, CSS, and JavaScript on a local XAMPP server. It successfully replaces the traditional manual library process with a fast, digital platform where the Admin can manage books with automatic per-copy LIB numbering, register members, issue and return books with live status updates, and monitor the entire library. Role-based access control, PHP session security, and SQL injection prevention ensure the system is robust and reliable.

Through comprehensive unit, integration, and system testing with 20 formal test cases, the system was verified to function correctly across all normal and error scenarios. The unique per-copy LIB numbering system provides a level of traceability not found in simpler library solutions. Overall, this project demonstrates the practical application of web technologies to solve a real-world institutional problem, and serves as a strong foundation for future enhancements such as online reservations, automatic fine calculation, SMS notifications, barcode integration, and a mobile application.

13. Appendix 1 – Coding

This appendix presents the important PHP code samples from the Library Management System project, organized by functionality.

13.1 config.php – Database Connection & LIB Number Generator

```
<?php
// Database connection settings
$host    = 'localhost';
$db_user = 'root';
$db_pass = '';
$db_name = 'library_system';

$conn = mysqli_connect($host, $db_user, $db_pass, $db_name);

if (!$conn) {
    die('Connection failed: ' . mysqli_connect_error());
}

// Generate next sequential LIB number
function nextLibNumber($conn) {
    $result = mysqli_query($conn,
        'SELECT book_number FROM book_copies
         ORDER BY copy_id DESC LIMIT 1');
    $row = mysqli_fetch_assoc($result);
    if (!$row) { return 'LIB001'; }
    $num = (int) substr($row['book_number'], 3);
    return 'LIB' . str_pad($num + 1, 3, '0', STR_PAD_LEFT);
}
?>
```

13.2 login.php – Login Validation and Session Creation

```
<?php
session_start();
require_once 'config.php';
$error = '';

if (isset($_POST['login'])) {
    $username = trim(mysqli_real_escape_string(
                    $conn, $_POST['username']));
    $password = trim(mysqli_real_escape_string(
                    $conn, $_POST['password']));

    if (empty($username) || empty($password)) {
        $error = 'Please enter username and password.';
    } else {
        $query  = "SELECT * FROM users
                   WHERE username = '$username'
```

```
                    AND password = '$password'";
        $result = mysqli_query($conn, $query);

        if (mysqli_num_rows($result) === 1) {
            $user = mysqli_fetch_assoc($result);
            $_SESSION['user_id']  = $user['id'];
            $_SESSION['username'] = $user['username'];
            $_SESSION['name']     = $user['name'];
            $_SESSION['role']     = $user['role'];
            if ($user['role'] === 'admin') {
                header('Location: admin.php');
            } else {
                header('Location: member.php');
            }
            exit();
        } else {
            $error = 'Invalid username or password.';
        }
    }
}
?>
```

13.3 Role-Based Access Control (top of every page)

```
<?php
session_start();
require_once 'config.php';

// For admin pages:
if (!isset($_SESSION['user_id']) ||
    $_SESSION['role'] !== 'admin') {
    header('Location: login.php');
    exit();
}

// For member pages:
if (!isset($_SESSION['user_id']) ||
    $_SESSION['role'] !== 'member') {
    header('Location: login.php');
    exit();
}
```

13.4 Add Book with Auto LIB Number Generation

```
if (isset($_POST['add_book'])) {
    $title  = trim(mysqli_real_escape_string(
              $conn, $_POST['title']));
    $author = trim(mysqli_real_escape_string(
              $conn, $_POST['author']));
    $pub    = trim(mysqli_real_escape_string(
              $conn, $_POST['publisher']));
    $qty    = (int)$_POST['quantity'];

    if (empty($title)||empty($author)||
        empty($pub)||$qty < 1) {
        $error = 'All fields required. Qty >= 1.';
    } else {
        // Insert book title
        mysqli_query($conn,
            "INSERT INTO books (title,author,publisher)
             VALUES ('$title','$author','$pub')");
        $book_id = mysqli_insert_id($conn);

        // Generate one copy record per quantity
        $generated = [];
        for ($i = 0; $i < $qty; $i++) {
            $lib = nextLibNumber($conn);
            mysqli_query($conn,
                "INSERT INTO book_copies
                 (book_id, book_number)
                 VALUES ($book_id,'$lib')");
            $generated[] = $lib;
        }
        $success = 'Book added. LIB: '
                 . implode(', ', $generated);
    }
}
```

13.5 Edit Book – Update Details and Adjust Copies

```
if (isset($_POST['edit_book'])) {
    $edit_id  = (int)$_POST['edit_id'];
    $title    = trim(mysqli_real_escape_string(
                $conn, $_POST['title']));
    $author   = trim(mysqli_real_escape_string(
                $conn, $_POST['author']));
    $pub      = trim(mysqli_real_escape_string(
                $conn, $_POST['publisher']));
    $new_total = (int)$_POST['new_total'];

    // Update book details
```

```
    mysqli_query($conn,
        "UPDATE books
         SET title='$title', author='$author',
         publisher='$pub'
         WHERE book_id=$edit_id");

    // Get current and issued copy counts
    $cur = (int)mysqli_fetch_assoc(mysqli_query($conn,
        "SELECT COUNT(*) c FROM book_copies
         WHERE book_id=$edit_id"))['c'];
    $iss = (int)mysqli_fetch_assoc(mysqli_query($conn,
        "SELECT COUNT(*) c FROM book_copies
         WHERE book_id=$edit_id
         AND status='issued'"))['c'];
    $diff = $new_total - $cur;

    if ($diff > 0) { // Add copies
        for ($i = 0; $i < $diff; $i++) {
            $lib = nextLibNumber($conn);
            mysqli_query($conn,
                "INSERT INTO book_copies
                 (book_id, book_number)
                 VALUES ($edit_id,'$lib')");
        }
    } elseif ($diff < 0) { // Remove copies
        if ($new_total < $iss) {
            $error = 'Cannot reduce below issued count.';
        } else {
            $rem  = abs($diff);
            $rows = mysqli_query($conn,
                "SELECT copy_id FROM book_copies
                 WHERE book_id=$edit_id
                 AND status='available'
                 ORDER BY copy_id DESC
                 LIMIT $rem");
            while ($rc = mysqli_fetch_assoc($rows)) {
                mysqli_query($conn,
                    "DELETE FROM book_copies
                     WHERE copy_id={$rc['copy_id']}");
            }
        }
    }
    if (!$error) $success = 'Book updated.';
}
```

6 Issue Book Query

```
if (isset($_POST['issue_book'])) {
    $user_id    = (int)$_POST['user_id'];
    $copy_id    = (int)$_POST['copy_id'];
    $issue_date = mysqli_real_escape_string(
                  $conn, $_POST['issue_date']);

    // Verify copy is available
    $chk = mysqli_fetch_assoc(mysqli_query($conn,
        "SELECT status, book_id FROM book_copies
         WHERE copy_id=$copy_id"));

    if ($chk['status'] !== 'available') {
        $error = 'This copy is already issued.';
    } else {
        $book_id = $chk['book_id'];

        // Check for duplicate title with same member
        $dup = mysqli_fetch_assoc(mysqli_query($conn,
            "SELECT COUNT(*) c
             FROM issued_books ib
             JOIN book_copies bc
             ON ib.copy_id = bc.copy_id
             WHERE ib.user_id=$user_id
             AND bc.book_id=$book_id
             AND ib.status='issued'"))['c'];

        if ($dup > 0) {
            $error = 'Member already holds this title.';
        } else {
            mysqli_query($conn,
                "INSERT INTO issued_books
                 (user_id, copy_id, issue_date,
                  status)
                 VALUES ($user_id, $copy_id,
                 '$issue_date', 'issued')");
            mysqli_query($conn,
                "UPDATE book_copies
                 SET status='issued'
                 WHERE copy_id=$copy_id");
            $success = 'Book issued successfully.';
        }
    }
}
```

13.7 Return Book Query

```
if (isset($_POST['return_book'])) {
    $issue_id    = (int)$_POST['issue_id'];
    $return_date = mysqli_real_escape_string(
                   $conn, $_POST['return_date']);

    $rec = mysqli_fetch_assoc(mysqli_query($conn,
        "SELECT * FROM issued_books
         WHERE issue_id=$issue_id
         AND status='issued'"));

    if (!$rec) {
        $error = 'No active issue found for this ID.';
    } else {
        $copy_id = $rec['copy_id'];
        mysqli_query($conn,
            "UPDATE issued_books
             SET status='returned',
             return_date='$return_date'
             WHERE issue_id=$issue_id");
        mysqli_query($conn,
            "UPDATE book_copies
             SET status='available'
             WHERE copy_id=$copy_id");
        $success = 'Book returned successfully.';
    }
}
```

13.8 Delete Book with Active Issue Protection

```
if (isset($_GET['delete'])) {
    $del_id = (int)$_GET['delete'];

    $active = mysqli_fetch_assoc(mysqli_query($conn,
        "SELECT COUNT(*) c
         FROM issued_books ib
         JOIN book_copies bc
         ON ib.copy_id = bc.copy_id
         WHERE bc.book_id=$del_id
         AND ib.status='issued'"))['c'];

    if ($active > 0) {
        $error = 'Cannot delete: ' . $active
               . ' copies are currently issued.';
    } else {
        mysqli_query($conn,
            "DELETE FROM books
             WHERE book_id=$del_id");
        $success = 'Book deleted successfully.';
```

```
    }
}
```

13.9 Fetch Books with Copy Statistics (SQL Query)

```
$books = mysqli_query($conn, "
    SELECT b.*,
      COUNT(bc.copy_id)          AS total_copies,
      SUM(bc.status='available') AS available_copies,
      SUM(bc.status='issued')    AS issued_copies,
      MIN(bc.book_number)        AS first_lib,
      MAX(bc.book_number)        AS last_lib
    FROM books b
    LEFT JOIN book_copies bc
    ON b.book_id = bc.book_id
    GROUP BY b.book_id
    ORDER BY b.book_id
");
```

13.10 logout.php – Session Destroy

```
<?php
session_start();
session_unset();
session_destroy();
header('Location: login.php');
exit();
?>
```

13.11 Database Schema (library_system.sql)

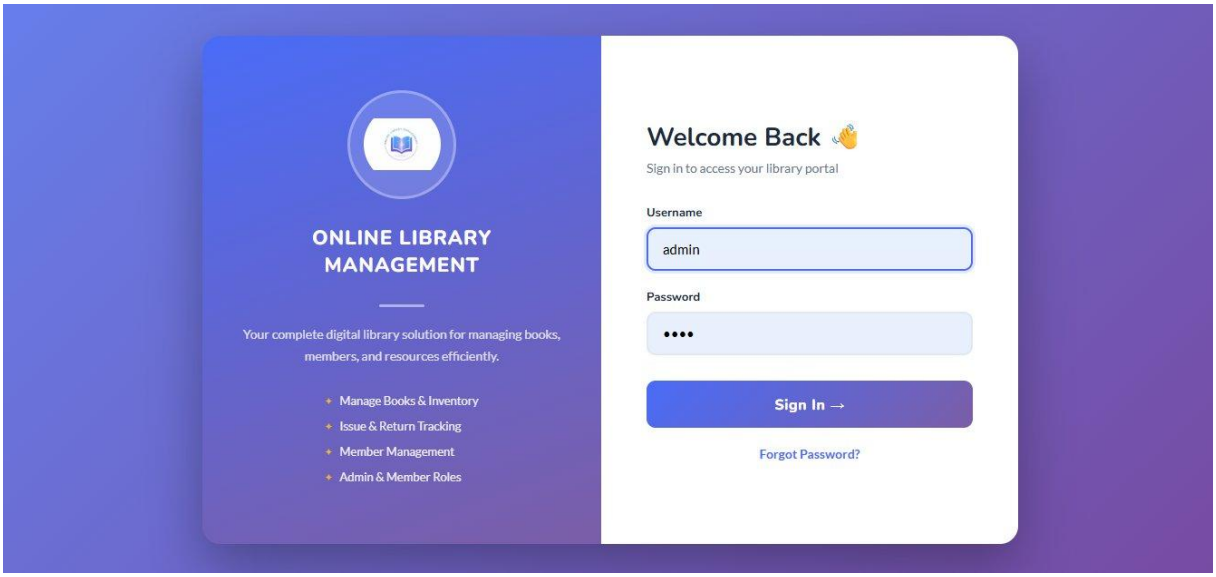
```
CREATE DATABASE IF NOT EXISTS library_system;
USE library_system;

CREATE TABLE users (
  id         INT AUTO_INCREMENT PRIMARY KEY,
  admin_id   VARCHAR(50)  DEFAULT NULL,
  reg_no     VARCHAR(50)  DEFAULT NULL,
  name       VARCHAR(100) NOT NULL,
  username   VARCHAR(50)  NOT NULL UNIQUE,
  password   VARCHAR(255) NOT NULL,
  role       ENUM('admin','member') NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE books (
  book_id    INT AUTO_INCREMENT PRIMARY KEY,
  title      VARCHAR(200) NOT NULL,
  author     VARCHAR(100) NOT NULL,
  publisher  VARCHAR(100) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE book_copies (
  copy_id     INT AUTO_INCREMENT PRIMARY KEY,
  book_id     INT NOT NULL,
  book_number VARCHAR(10) NOT NULL UNIQUE,
  status      ENUM('available','issued')
              DEFAULT 'available',
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (book_id)
    REFERENCES books(book_id) ON DELETE CASCADE
);

CREATE TABLE issued_books (
  issue_id    INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT NOT NULL,
  copy_id     INT NOT NULL,
  issue_date  DATE NOT NULL,
  return_date DATE DEFAULT NULL,
  status      ENUM('issued','returned')
              DEFAULT 'issued',
  FOREIGN KEY (user_id)
    REFERENCES users(id) ON DELETE CASCADE,
  FOREIGN KEY (copy_id)
    REFERENCES book_copies(copy_id) ON DELETE CASCADE
);

-- Default admin account
INSERT INTO users
  (name, username, password, role)
VALUES
  ('Administrator','admin','admin123','admin');
```

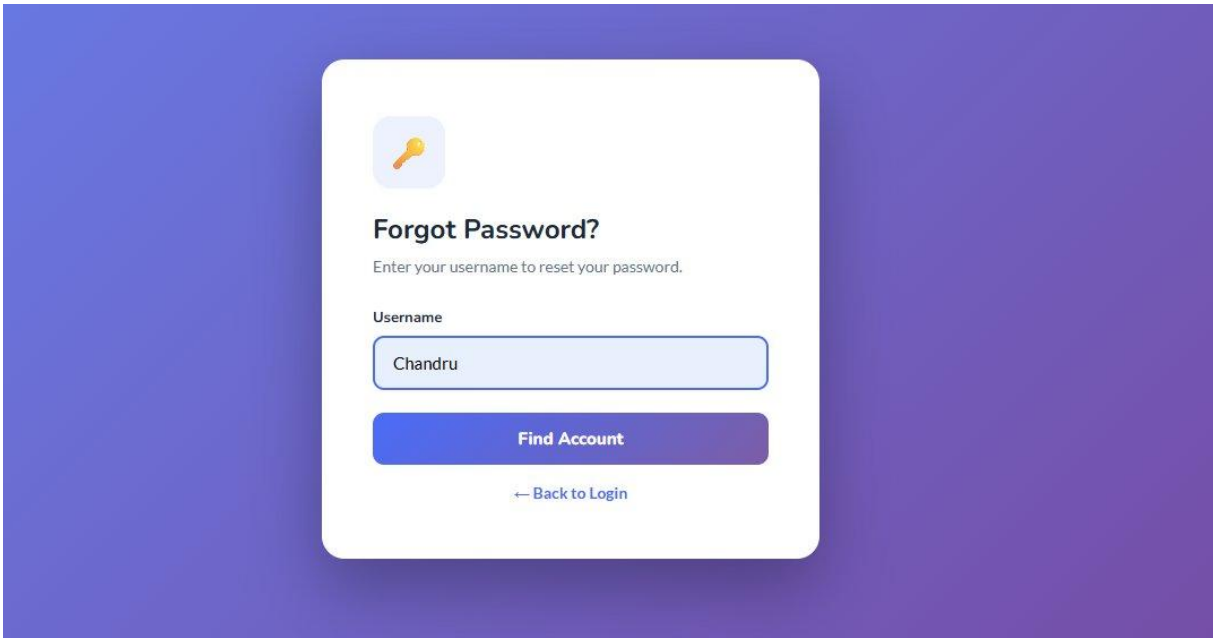
14. Appendix 2 – Screenshot

The following screenshots show the main pages of the Library Management System as they appear in the browser during actual use.

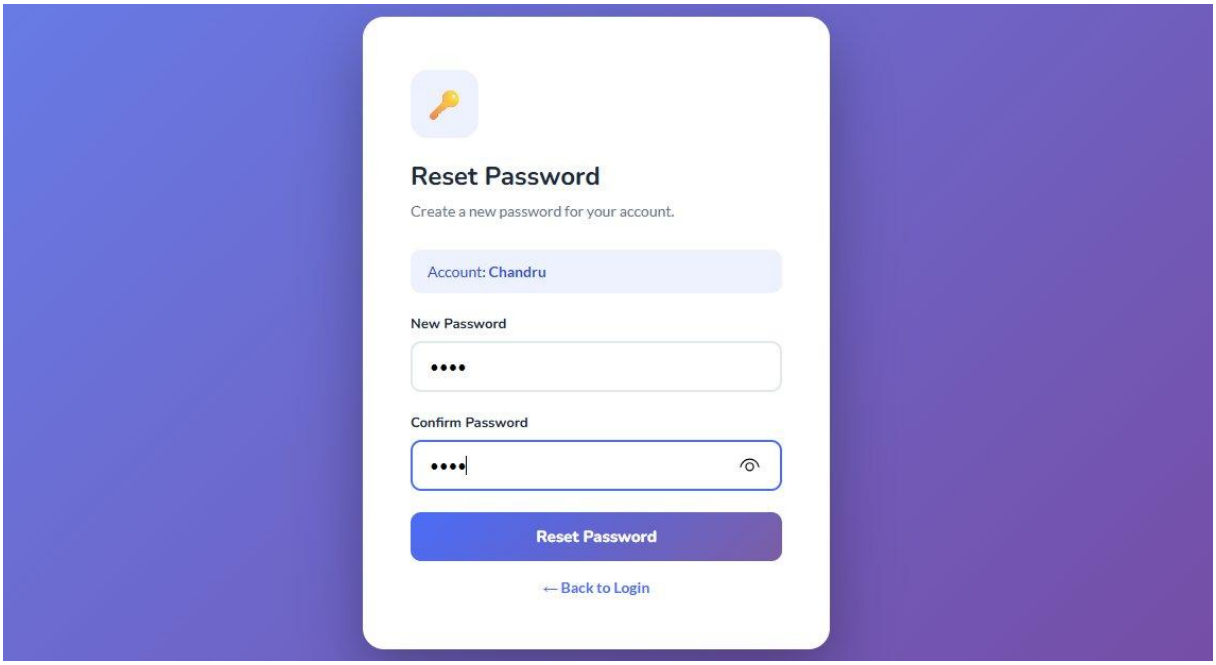
Screenshot 1: Login Page



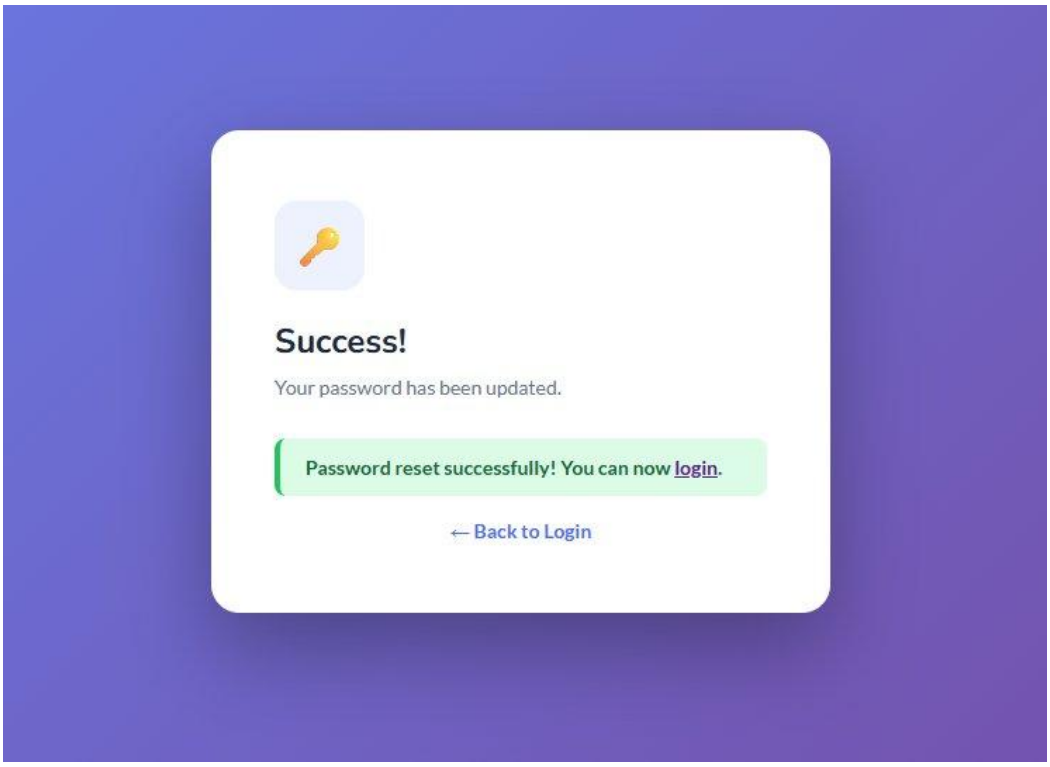
Screenshot 2: Forgot Password



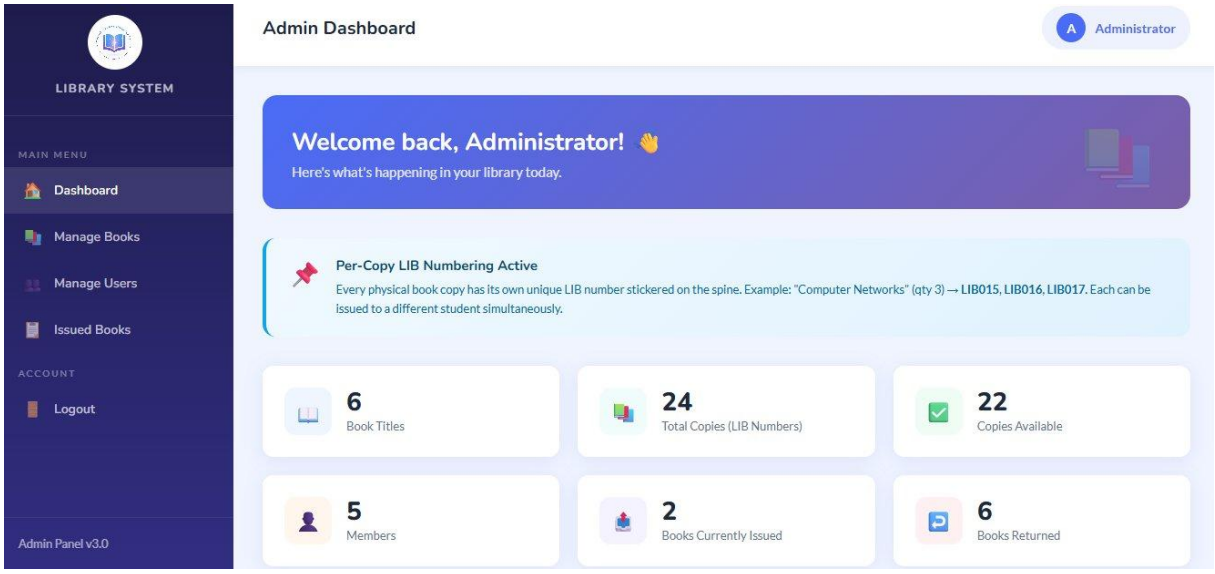
Screenshot 3: Reset Password



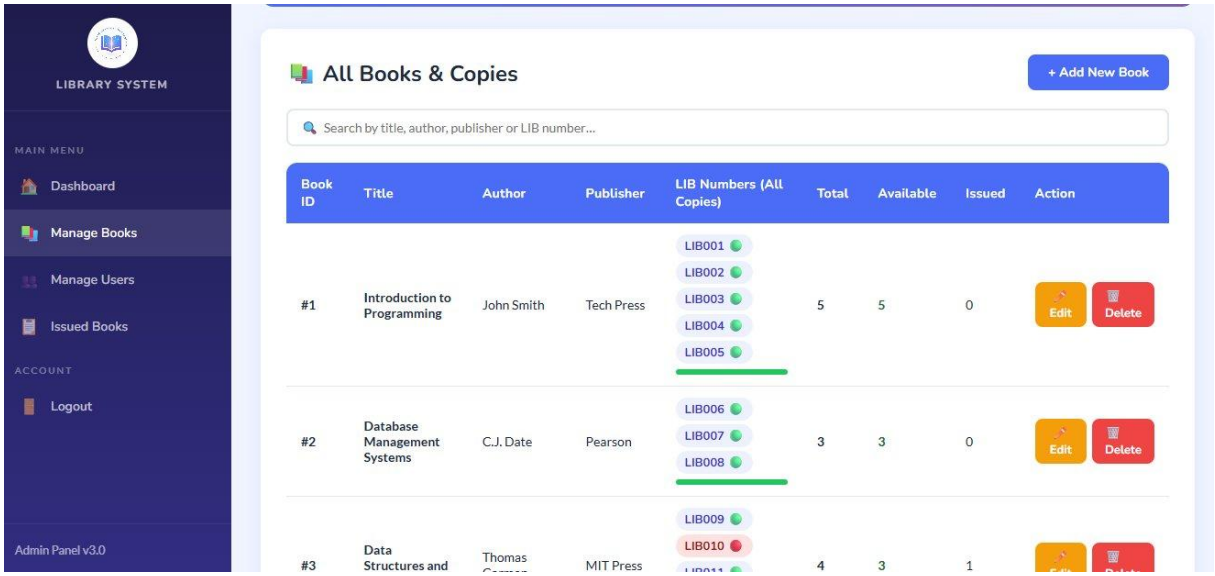
Screenshot 4: Password Reset – Success



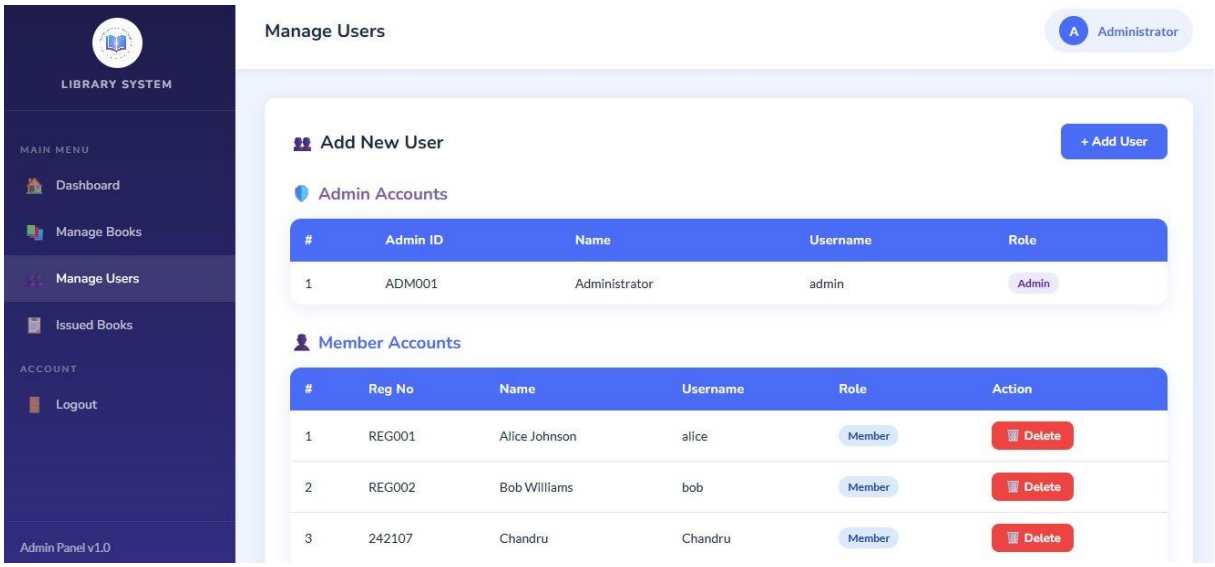
Screenshot 5: Admin Dashboard



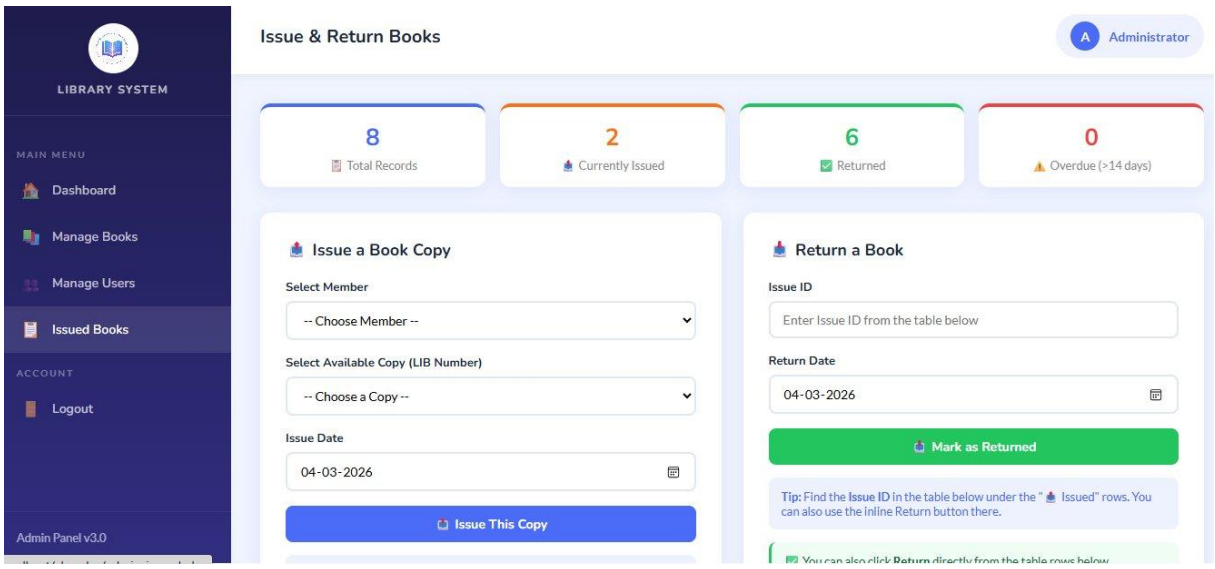
Screenshot 6: Manage Books – All Books & Copies



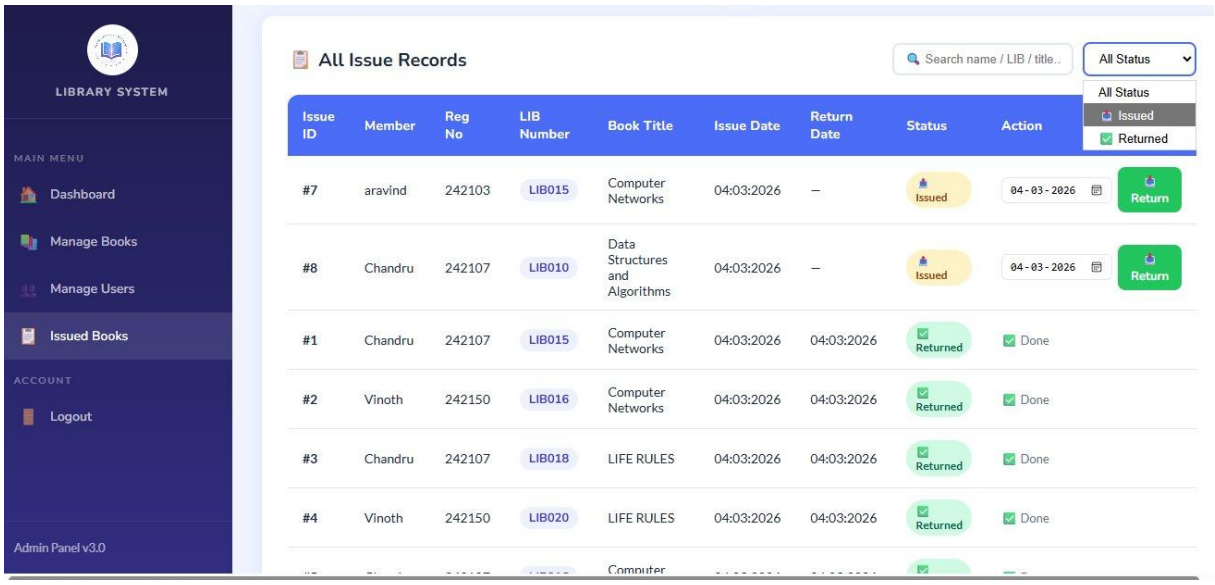
Screenshot 7: Manage Users



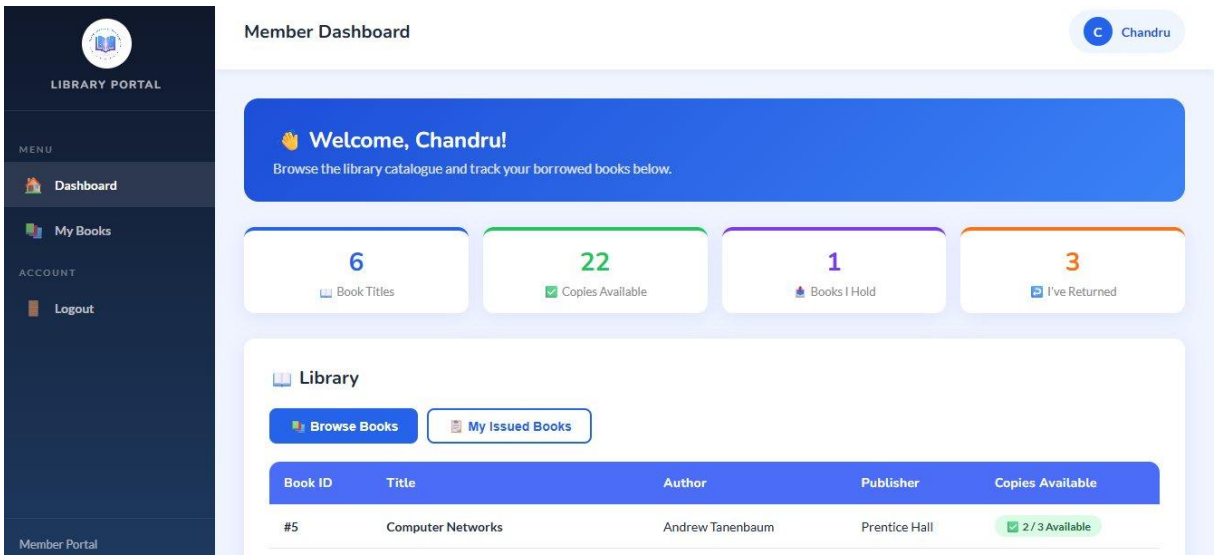
Screenshot 8: Issue & Return Books



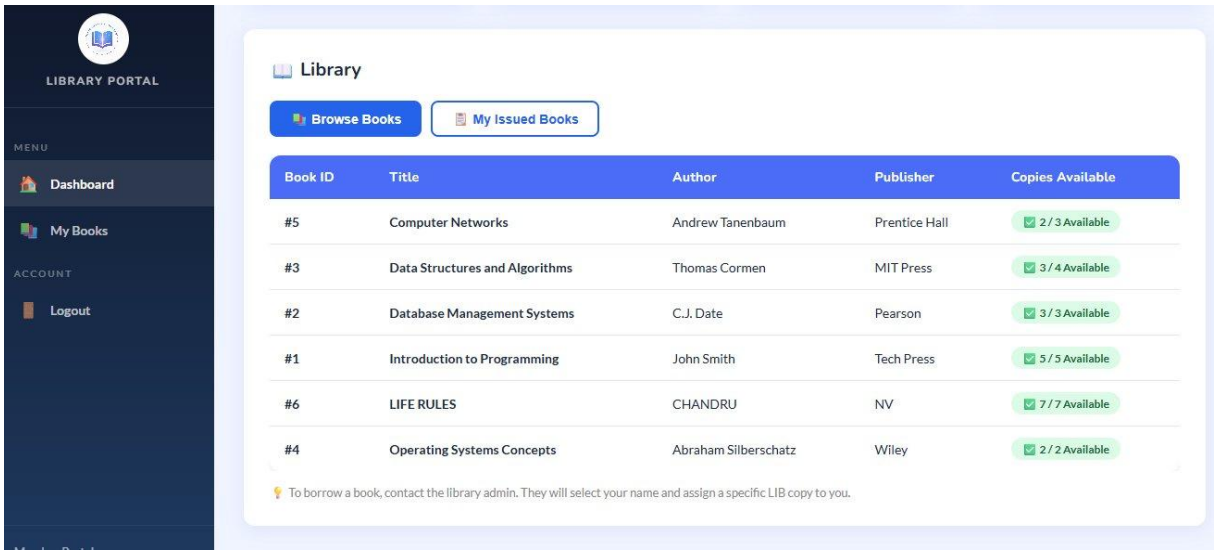
Screenshot 9: All Issue Records



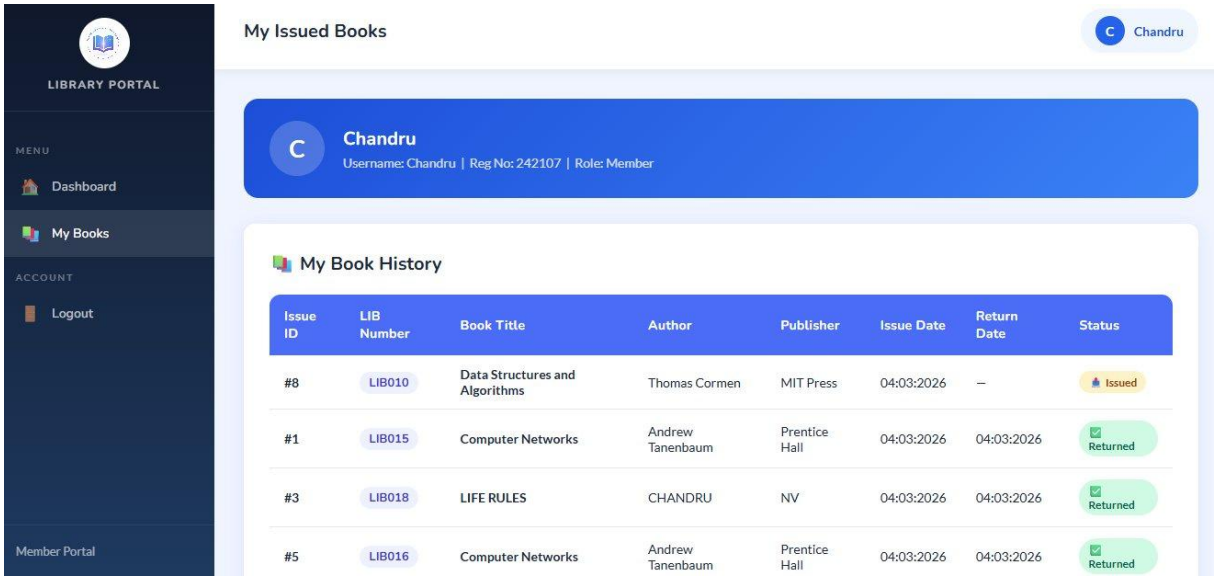
Screenshot 10: Member Dashboard



Screenshot 11: Member – Browse Books



Screenshot 12: Member – My Issued Books



15. Bibliography

The following references, textbooks, and online resources were consulted during the design, development, and documentation of the Library Management System project.

Online References

1. PHP Official Documentation – www.php.net. Comprehensive reference for PHP functions, session handling, form processing, and database connectivity used throughout the project.
2. MySQL Official Documentation – www.mysql.com. Reference for SQL syntax, table design, data types, foreign keys, and aggregate functions used in the database.
3. W3Schools Web Tutorials – www.w3schools.com. Used as a quick reference for HTML5, CSS3, JavaScript, and PHP examples during development of the user interface.
4. XAMPP Official Site – www.apachefriends.org. Reference for setting up and configuring the local Apache, MySQL, and PHP development environment.
5. MDN Web Docs – developer.mozilla.org. Reference for JavaScript DOM manipulation, event handling, and CSS properties used in live search and modal functionality.

Textbooks

6. Luke Welling and Laura Thomson, PHP and MySQL Web Development, 5th Edition, Addison-Wesley, 2016. Primary reference for PHP-MySQL integration, form handling, and session management.
7. Robin Nixon, Learning PHP, MySQL, JavaScript, CSS & HTML5, 6th Edition, O'Reilly Media, 2021. Supplementary reference for full-stack web development techniques.
8. Ramez Elmasri and Shamkant Navathe, Fundamentals of Database Systems, 7th Edition, Pearson Education, 2016. Reference for database design, entity-relationship modeling, and normalization concepts.

Tools and Software Used

- XAMPP v3.3.0 – Local server environment providing Apache, MySQL, and PHP.
- Visual Studio Code – Primary code editor used for writing PHP, HTML, CSS, and JavaScript.
- phpMyAdmin – Used for creating and managing the MySQL database visually during development.

- Google Chrome (Version 120+) – Primary browser used for testing the web application.
- Mozilla Firefox – Used for cross-browser compatibility testing.
- Microsoft Edge – Used for additional cross-browser compatibility verification.